Федеральное государственное образовательное бюджетное учреждение
высшего образования
«Финансовый университет при правительстве
Российской Федерации»

(Финансовый университет)

Колледж информатики и программирования

Оптимизация веб приложений

Группа: 4ИСИП-1023

Лабораторная работа №8

На тему: Настройка Apache + PHP-FPM в VirtualBox с SSH-доступом, ЧПУ и динамическим каталогом

Выполнил(а) работу

_____К.Э.Бадыкина

Принял

_____А.С.Рыбаков

Москва

2026

Цель работы: Изучить принципы работы веб-сервера Apache в связке с менеджером процессов PHP-FPM. Освоить конфигурирование виртуальной машины в VirtualBox с организацией удалённого SSH-доступа через механизм проброса портов (Port Forwarding). Научиться настраивать Apache в режиме обратного прокси для обработки динамических запросов через Unix-сокет PHP-FPM, подключать модуль mod_rewrite и составлять конфигурационные директивы .htaccess для генерации человекопонятных URL (ЧПУ). Разработать прототип динамического каталога магазина на PHP и SQLite с маршрутизацией по слагам и поддержкой интерфейса добавления товаров.

Теоретическая справка

2.1. Взаимодействие Apache и PHP-FPM

В отличие от классического модуля mod_php, встроенного напрямую в каждый процесс сервера, современная модель использует PHP-FPM (FastCGI Process Manager) – независимый сервис, управляющий пулом рабочих процессов PHP.

- Веб-сервер Apache принимает клиентский HTTP-запрос.
- Если запрашивается файл с расширением .php, директива SetHandler перенаправляет запрос через модуль proxy_fcgi в сокет PHP-FPM по протоколу FastCGI.
- В операционной системе обмен данными происходит через локальный Unix Domain Socket (в данной работе: /run/php/php8.5-fpm.sock), что минимизирует сетевые издержки.
- PHP-FPM выполняет байт-код и возвращает сгенерированный контент Apache, который передаёт его клиенту в теле ответа.

2.2. Механизм mod_rewrite и формирование ЧПУ

Модуль mod_rewrite представляет собой механизм преобразования URL на основе правил сопоставления регулярных выражений.

- При включении директивы AllowOverride All в конфигурации виртуального хоста Apache считывает локальные файлы .htaccess.
- С помощью правил RewriteRule и условий RewriteCond сервер сопоставляет путь запроса (например, /catalog/electronics/smartphone-x) и транслирует его во внутренний вызов скрипта (например,

product.php?category_slug=electronics&product_slug=smartphone-x) без изменения адреса в браузере пользователя.

– Флаг [L] (Last) завершает текущий цикл перезаписи при совпадении правила, а [QSA] (Query String Append) сохраняет исходные GET-параметры запроса.

2.3. Сетевая архитектура VirtualBox (NAT и Port Forwarding)

При типе сетевого адаптера NAT виртуальная машина находится за внутренней таблицей трансляции адресов гипервизора и изолирована от внешней локальной сети. Чтобы обеспечить доступ с хост-системы:

– Настраивается проброс портов (Port Forwarding): пакеты, поступающие на назначенный порт хоста (например, 127.0.0.1:2222 и 127.0.0.1:8080), транслируются виртуальным маршрутизатором на порты гостевой ОС (:22 для SSH и :80 для Apache).

– Порт 80 хоста не используется, чтобы исключить конфликты с локальными сетевыми службами основной операционной системы.

3. Листинги конфигураций

Конфигурация виртуального хоста Apache, которая связывает сервер с PHP-FPM и разрешает использование .htaccess, представлена в Листинге 1.

Листинг 1 – Конфигурация VirtualHost

```
<VirtualHost *:80>
    ServerName localhost
    DocumentRoot /var/www/shop
    <Directory /var/www/shop>
        Options Indexes FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>
    <FilesMatch \.php$>
        SetHandler "proxy:unix:/run/php/php8.3-fpm.sock|fcgi://localhost"
    </FilesMatch>
    ErrorLog ${APACHE_LOG_DIR}/shop_error.log
    CustomLog ${APACHE_LOG_DIR}/shop_access.log combined
</VirtualHost>
```

Механизм преобразования «красивых» ссылок во внутренние запросы к PHP-скриптам реализован с помощью правил mod_rewrite, приведенных в Листинге 2.

Листинг 2 – Файл .htaccess

```
# Включаем механизм перезаписи URL
RewriteEngine On
# Устанавливаем базовый путь для правил
RewriteBase /
```

```
# Условие 1: если запрошен реальный файл (-f)
# Условие 2 (OR): если запрошен реальный каталог (-d)
RewriteCond %{REQUEST_FILENAME} -f [OR]
RewriteCond %{REQUEST_FILENAME} -d
# Тогда ничего не переписываем и отдаем как есть, [L] - остановить обработку
RewriteRule ^ - [L]

# Админка: добавление товара
RewriteRule ^admin/add-product/?$ admin_add_product.php [L,QSA]

# Главная страница каталога
RewriteRule ^catalog/?$ index.php [L,QSA]

# Категория: /catalog/electronics
# ([a-z0-9-]+) - захватываем слаг категории в переменную $1
# [QSA] - сохраняет исходные GET-параметры (если были)
RewriteRule ^catalog/([a-z0-9-]+)/?$ category.php?slug=$1 [L,QSA]

# Товар: /catalog/electronics/smartphone-x
# Первая скобка - слаг категории ($1), вторая - слаг товара ($2)
RewriteRule ^catalog/([a-z0-9-]+)/([a-z0-9-]+)/?$
product.php?category_slug=$1&product_slug=$2 [L,QSA]
```

Структура таблиц для хранения категорий и товаров, а также их начальное наполнение, показаны в Листинге 3.

Листинг 3 – SQL-схема базы данных SQLite

```
-- Таблица категорий
CREATE TABLE categories (
    id INTEGER PRIMARY KEY,      -- Уникальный идентификатор категории
    slug TEXT UNIQUE,            -- Человекопонятный URL-идентификатор
(уникальный)
    name TEXT                    -- Отображаемое название категории
);

-- Таблица товаров
CREATE TABLE products (
    id INTEGER PRIMARY KEY,      -- Уникальный идентификатор товара
    category_id INTEGER,         -- Внешний ключ на таблицу категорий
    slug TEXT,                   -- Человекопонятный URL-идентификатор товара
    name TEXT,                   -- Название товара
    price REAL,                  -- Цена
    description TEXT,            -- Описание товара
    FOREIGN KEY(category_id) REFERENCES categories(id) -- Связь с категорией
);

-- Наполнение тестовыми данными
INSERT INTO categories (slug, name) VALUES
('electronics', 'Электроника'),
('books', 'Книги');

INSERT INTO products (category_id, slug, name, price, description) VALUES
(1, 'smartphone-x', 'Смартфон X', 29999.00, 'Мощный смартфон с отличной
камерой'),
(1, 'laptop-pro', 'Ноутбук Pro', 74999.00, 'Ноутбук для работы и творчества'),
(2, 'php-book', 'PHP для профессионалов', 1500.00, 'Лучшая книга по PHP');
```

Логика вывода данных реализована в PHP-скриптах. Скрипт вывода списка категорий представлен в Листинге 4.

Листинг 4 – Главная страница (index.php)

```
<?php
// Подключение к базе данных SQLite
$db = new PDO('sqlite:' . __DIR__ . '/data/shop.db');

// Получение всех категорий из БД
```

```
$categories = $db->query("SELECT * FROM categories")-
>fetchAll(PDO::FETCH_ASSOC);
?>
<h2>Каталог магазина</h2>
<ul>
<?php foreach ($categories as $cat): ?>
    <li>
        <!-- Формирование ЧПУ-ссылки на категорию -->
        <a href="/catalog/<?= htmlspecialchars($cat['slug']) ?>">
            <!-- htmlspecialchars защищает от XSS-атак при выводе названия -->
            <?= htmlspecialchars($cat['name']) ?>
        </a>
    </li>
<?php endforeach; ?>
</ul>
<p><a href="/admin/add-product">+ Добавить товар</a></p>
```

Скрипт вывода товаров конкретной категории с использованием подготовленных запросов приведен в Листинге 5.

Листинг 5 – Страница категории (category.php)

```
<?php
// Подключение к БД
$db = new PDO('sqlite:' . __DIR__ . '/data/shop.db');

// Получение слага категории из GET-запроса (переданного через .htaccess)
$slug = $_GET['slug'] ?? '';

// Поиск категории по слагу (используем подготовленные выражения для защиты от
SQL-инъекций)
$stmt = $db->prepare("SELECT * FROM categories WHERE slug = ?");
$stmt->execute([$slug]);
$category = $stmt->fetch(PDO::FETCH_ASSOC);

// Если категория не найдена - возвращаем 404
if (!$category) {
    http_response_code(404);
    die('Категория не найдена');
}

// Получение товаров, принадлежащих этой категории
$stmt = $db->prepare("SELECT * FROM products WHERE category_id = ?");
$stmt->execute([$category['id']]);
$products = $stmt->fetchAll(PDO::FETCH_ASSOC);
?>
<h2><?= htmlspecialchars($category['name']) ?></h2>
<ul>
<?php foreach ($products as $p): ?>
    <li>
        <!-- Ссылка на товар: /catalog/категория/товар -->
        <a href="/catalog/<?= htmlspecialchars($slug) ?>/<?=
htmlspecialchars($p['slug']) ?>">
            <!-- Форматирование цены с пробелами (number_format) -->
            <?= htmlspecialchars($p['name']) ?> — <?=
number_format($p['price'], 0, '', ' ') ?> ₽
        </a>
    </li>
<?php endforeach; ?>
</ul>
<p><a href="/">Назад к категориям</a></p>
```

Скрипт карточки отдельного товара, использующий объединение таблиц (JOIN), описан в Листинге 6.

Листинг 6 – Карточка товара (product.php)

```
<?php
// Подключение к БД
$db = new PDO('sqlite:' . __DIR__ . '/data/shop.db');

// Получение слагов из GET-запроса
$catSlug = $_GET['category_slug'] ?? '';
$prodSlug = $_GET['product_slug'] ?? '';

// Сложный запрос с JOIN: получаем товар и имя его категории по двум слагам
$stmt = $db->prepare("
    SELECT p.*, c.name as category_name, c.slug as category_slug
    FROM products p
    JOIN categories c ON p.category_id = c.id
    WHERE c.slug = ? AND p.slug = ?
");
$stmt->execute([$catSlug, $prodSlug]);
$product = $stmt->fetch(PDO::FETCH_ASSOC);

// Если товар не найден - 404
if (!$product) {
    http_response_code(404);
    die('Товар не найден');
}
?>
<h2><?= htmlspecialchars($product['name']) ?></h2>
<p><strong>Категория:</strong> <?= htmlspecialchars($product['category_name'])
?></p>
<p><strong>Цена:</strong> <?= number_format($product['price'], 0, '', ' ') ?>
₽</p>
<p><?= htmlspecialchars($product['description']) ?></p>
<p>
    <!-- Ссылки для навигации -->
    <a href="/catalog/<?= htmlspecialchars($product['category_slug'])
?>">Назад в категорию</a> |
    <a href="/">На главную</a>
</p>
```

Скрипт обработки формы добавления нового товара, включающий валидацию и PRG-паттерн, представлен в Листинге 7.

Листинг 7 – Страница добавления товара (admin_add_product.php)

```
<?php
// Подключение к БД и включение режима исключений
$db = new PDO('sqlite:' . __DIR__ . '/data/shop.db');
$db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

$errors = [];
// Получаем список категорий для выпадающего списка
$categories = $db->query("SELECT * FROM categories ORDER BY name")-
>fetchAll(PDO::FETCH_ASSOC);

// Обработка формы при POST-запросе
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // Приведение и очистка входных данных
    $categoryId = (int)($_POST['category_id'] ?? 0);
    $name       = trim($_POST['name'] ?? '');
    $slug       = trim($_POST['slug'] ?? '');
    $price      = (float)($_POST['price'] ?? 0);
    $desc       = trim($_POST['description'] ?? '');

    // Валидация полей
    if ($categoryId <= 0) {
        $errors[] = 'Выберите категорию.';
    }
    if ($name === '') {
```

```
        $errors[] = 'Название товара обязательно.';
    }
    if ($slug === '') {
        $errors[] = 'Слаг (URL) обязателен.';
    } elseif (!preg_match('/^[a-z0-9-]+$/', $slug)) {
        // Проверка, что слаг состоит только из разрешенных символов
        $errors[] = 'Слаг может содержать только строчные латинские буквы,
цифры и дефис.';
    }
    if ($price <= 0) {
        $errors[] = 'Цена должна быть больше нуля.';
    }

    // Проверка на дубликат слага в рамках выбранной категории
    if (!$errors) {
        $stmt = $db->prepare("SELECT COUNT(*) FROM products WHERE category_id
= ? AND slug = ?");
        $stmt->execute([$categoryId, $slug]);
        if ($stmt->fetchColumn() > 0) {
            $errors[] = 'Товар с таким слагом уже есть в этой категории.';
        }
    }

    // Если ошибок нет - сохраняем в БД
    if (!$errors) {
        try {
            $stmt = $db->prepare("
                INSERT INTO products (category_id, slug, name, price,
description)
                VALUES (?, ?, ?, ?, ?)
            ");
            $stmt->execute([$categoryId, $slug, $name, $price, $desc]);

            // Получаем слаг категории для редиректа
            $stmt = $db->prepare("SELECT slug FROM categories WHERE id = ?");
            $stmt->execute([$categoryId]);
            $catSlug = $stmt->fetchColumn();

            // PRG-паттерн: редирект на карточку товара, чтобы избежать дублей
при F5
            header("Location: /catalog/{$catSlug}/{$slug}");
            exit;
        } catch (PDOException $e) {
            $errors[] = 'Ошибка БД: ' . $e->getMessage();
        }
    }
}
?>
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Добавить товар</title>
</head>
<body>
<h2>Добавить товар в каталог</h2>

<!-- Блок вывода ошибок валидации -->
<?php if ($errors): ?>
    <div style="color:red; border:1px solid red; padding:10px;">
        <ul>
            <?php foreach ($errors as $e): ?>
                <li><?= htmlspecialchars($e) ?></li>
            <?php endforeach; ?>
        </ul>
    </div>
<?php endif; ?>
```

```
<!-- HTML-форма. Значения сохраняются при ошибке через
htmlspecialchars($_POST[...]) -->
<form method="post" action="/admin/add-product">
    <p>
        <label>Категория:<br>
            <select name="category_id">
                <option value="0">— выберите —</option>
                <?php foreach ($categories as $cat): ?>
                    <option value="<?= $cat['id'] ?>"
                        <?= (($_POST['category_id'] ?? 0) == $cat['id']) ?
'selected' : '' ?>>
                        <?= htmlspecialchars($cat['name']) ?>
                    </option>
                <?php endforeach; ?>
            </select>
        </label>
    </p>
    <p>
        <label>Название:<br>
            <input type="text" name="name" value="<?=
htmlspecialchars($_POST['name'] ?? '') ?>" required>
        </label>
    </p>
    <p>
        <label>Слаг (URL):<br>
            <!-- Паттерн на клиенте дублирует проверку на сервере -->
            <input type="text" name="slug" value="<?=
htmlspecialchars($_POST['slug'] ?? '') ?>"
                   pattern="[a-z0-9-]+" title="Только строчные латинские
буквы, цифры и дефис" required>
        </label>
    </p>
    <p>
        <label>Цена (₽):<br>
            <input type="number" step="0.01" min="0.01" name="price"
                   value="<?= htmlspecialchars($_POST['price'] ?? '') ?>"
required>
        </label>
    </p>
    <p>
        <label>Описание:<br>
            <textarea name="description" rows="4" cols="50"><?=
htmlspecialchars($_POST['description'] ?? '') ?></textarea>
        </label>
    </p>
    <p>
        <button type="submit">Сохранить товар</button>
        <a href="/">Отмена</a>
    </p>
</form>
</body>
</html>
```

В ходе проверки работы веб-приложения были сделаны следующие скриншоты. Главная страница каталога со списком доступных категорий представлена на рисунке 1.

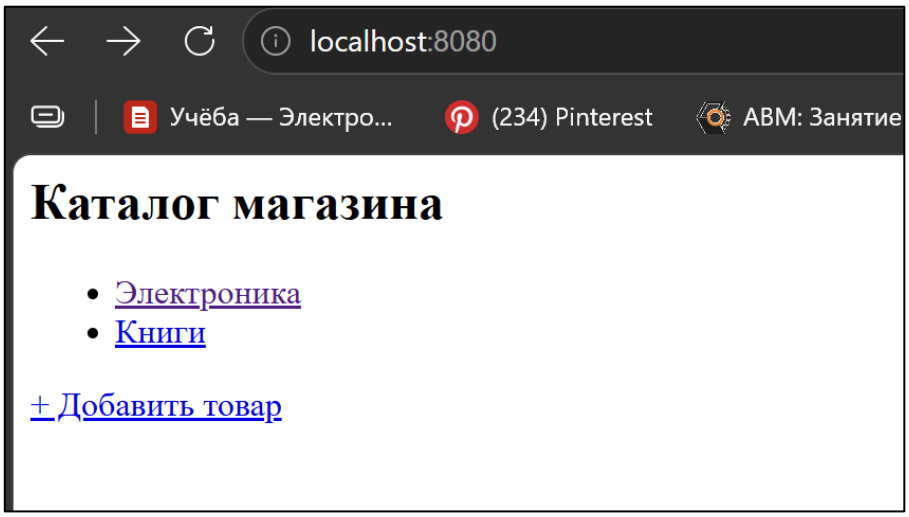


Рисунок 1 – Главная страница каталога

При переходе по ЧПУ-ссылке открывается страница выбранной категории, что продемонстрировано на рисунке 2.

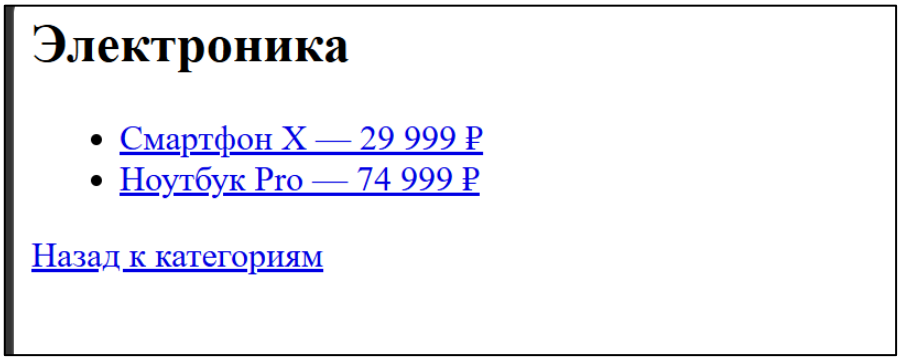


Рисунок 2 – Страница категории «Электроника» с товарами

Внешний вид карточки отдельного товара, открытой по ЧПУ-адресу, показан на рисунке 3.

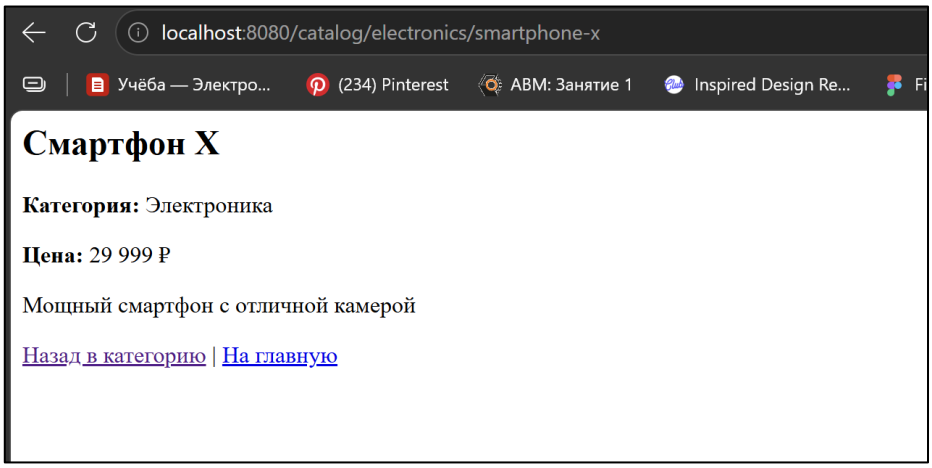


Рисунок 3 – Карточка товара «Смартфон X»

Процесс добавления нового товара через административную форму отображен на рисунке 4.

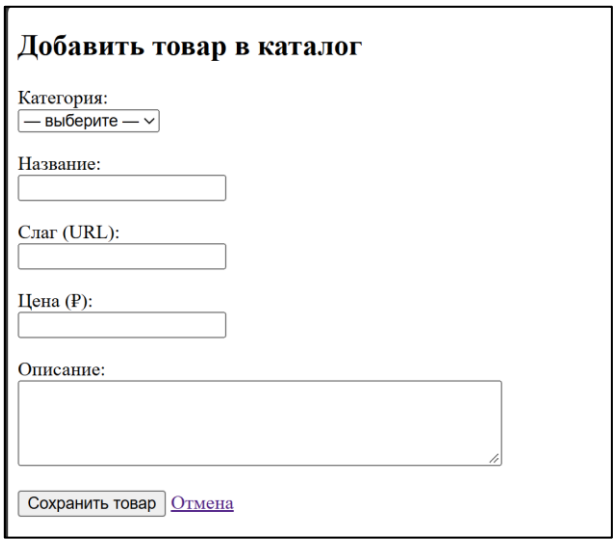


Рисунок 4 – Добавление нового товара в каталог

Работа серверной валидации при попытке добавить товар с уже существующим слагом показана на рисунке 5.

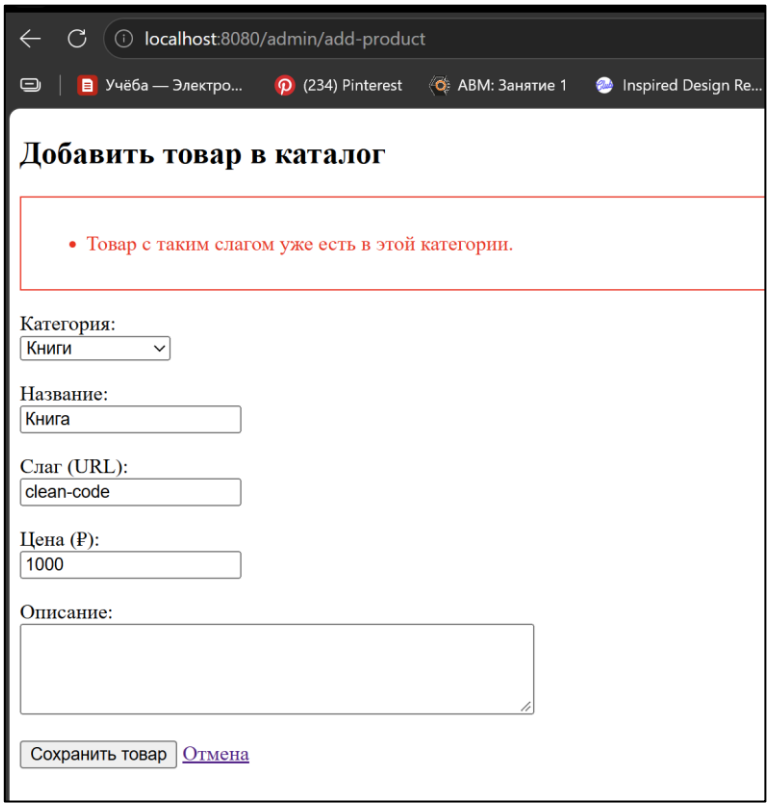


Рисунок 5 – Вывод сообщения об ошибке при дублировании слага

Результат успешного сохранения товара в базу данных и перенаправления на его карточку представлен на рисунке 6.

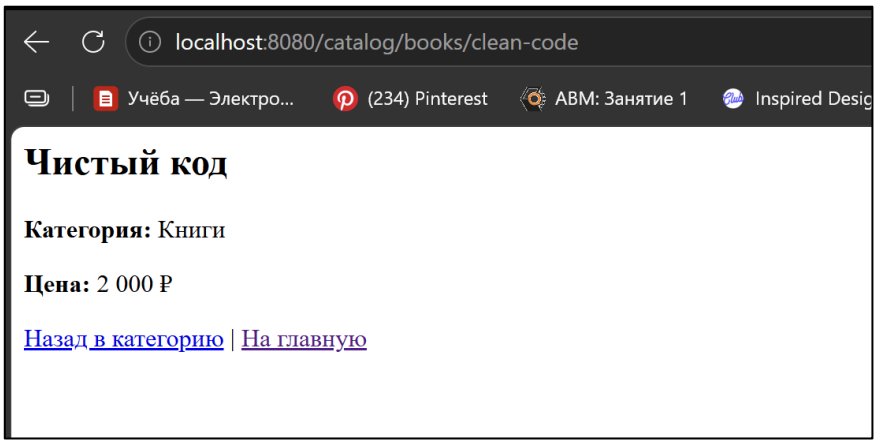


Рисунок 6 – Карточка успешно добавленного товара по ЧПУ-адресу

Корректность настройки конфигурации веб-сервера подтверждается выводом команды apachectl configtest, который приведен на рисунке 7.

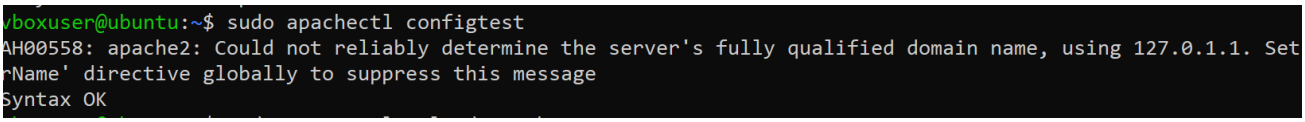


Рисунок 7 – Проверка конфигурации Apache

Наличие запущенного сокета PHP-FPM проверено с помощью команды ls /run/php/, результат которой отображен на рисунке 8.

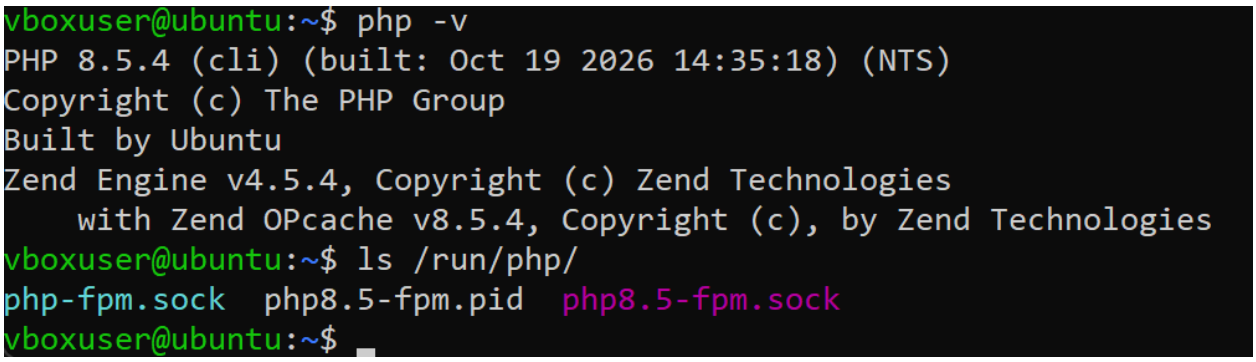


Рисунок 8 – Проверка наличия сокета PHP-FPM

Вывод: В ходе выполнения лабораторной работы я изучил принципы связки веб-сервера Apache и менеджера процессов PHP-FPM через Unix-сокет. Освоил настройку виртуальной машины VirtualBox с типом сети NAT и пробросом портов для доступа по SSH и HTTP. Научился настраивать виртуальные хосты Apache, включать модуль mod_rewrite и писать правила .htaccess для реализации человекопонятных URL. Также я получил практические навыки работы с базой данных SQLite через PDO в PHP, реализовал динамический каталог товаров и форму добавления записей с валидацией и защитой от повторной отправки формы.